

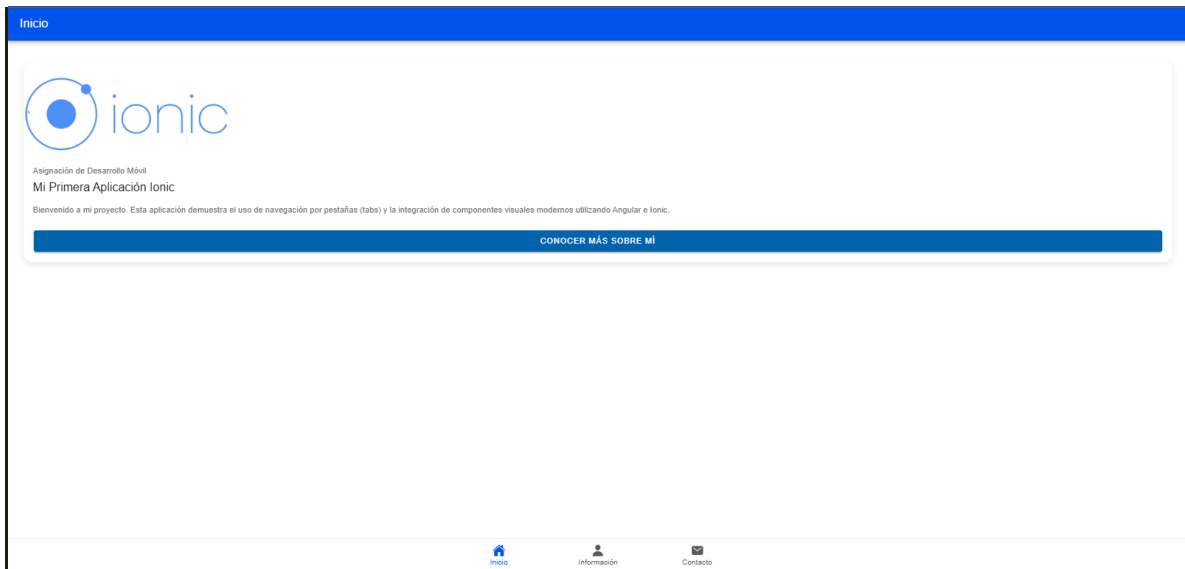


UNIVERSIDAD NACIONAL EXPERIMENTAL
PARA LAS TELECOMUNICACIONES E INFORMÁTICA
(UNETI) VICERRECTORADO ACADÉMICO
PROGRAMA NACIONAL DE FORMACIÓN EN
INFORMÁTICA

IONIC

INTEGRANTE
Yesmir Guzman
CI20130682

Caracas, Mayo del 2026



FRONTEND

```
<ion-header [translucent]="true">

  <ion-toolbar color="primary">
    <ion-title>
      Inicio
    </ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true" class="ion-padding">
  <!--
  He decidido usar un ion-card para la bienvenida porque le da un aspecto
  más profesional y organizado a la pantalla principal.
  Este componente agrupa visualmente la imagen y el texto.
  -->

  <ion-card class="welcome-card">
    <!--
    Esta es la imagen de bienvenida que generamos.
    La ruta apunta a la carpeta de assets del proyecto.
    -->

    <ion-card-header>
      <ion-card-subtitle>Asignación de Desarrollo Móvil</ion-card-subtitle>
      <ion-card-title>Mi Primera Aplicación Ionic</ion-card-title>
    </ion-card-header>
```

```

<ion-card-content>
  <!--
    Aquí explico brevemente el propósito de la app.
    He configurado tres secciones principales para cumplir con los requisitos.
  -->

  Bienvenido a mi proyecto. Esta aplicación demuestra el uso de navegación por pestañas (tabs)
  y la integración de componentes visuales modernos utilizando Angular e Ionic.

  <br><br>
  <ion-button expand="block" color="secondary" routerLink="/tabs/informacion">
    Conocer más sobre mí
  </ion-button>
</ion-card-content>
</ion-card>
</ion-content>

```

BACKEND

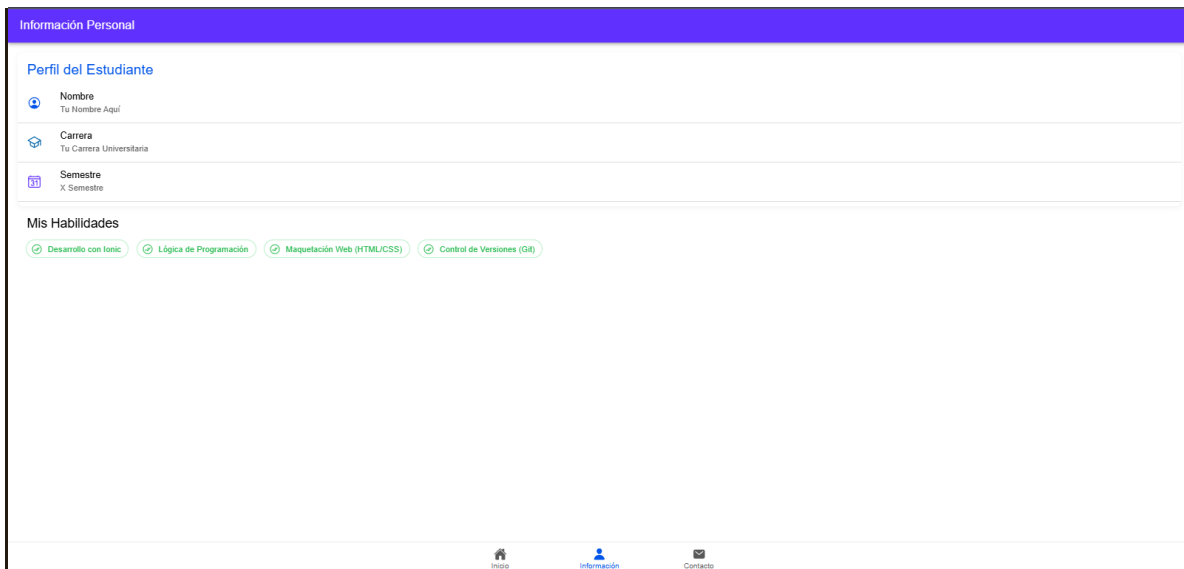
```

import { Component } from '@angular/core';

@Component({
  selector: 'app-inicio',
  templateUrl: 'inicio.page.html',
  styleUrls: ['inicio.page.scss'],
  standalone: false,
})
export class InicioPage {
  /**
   * esta clase representa la lógica de mi página de inicio.
   * El decorador @Component arriba es el que une este archivo TypeScript con mi archivo HTML.
   */

  constructor() {
    /**
     * El constructor es el primer método que se ejecuta al cargar la página.
     * Por ahora no necesito inicializar nada complejo, así que lo dejo limpio.
     */

    console.log('La página de Inicio se ha cargado correctamente.');
```



FRONTEND

```
<ion-header [translucent]="true">

  <ion-toolbar color="tertiary">

    <ion-title>
      Información Personal
    </ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true" class="ion-padding">
  <!--
    Para organizar mi información, he elegido un ion-list.
    Es la forma más limpia de mostrar datos estructurados en una app móvil.
  -->

  <ion-list lines="full" class="student-list">

    <ion-list-header>

      <ion-label color="primary"><h1>Perfil del Estudiante</h1></ion-label>
    </ion-list-header>

    <!--
      Aquí utilizo la "Interpolación" (las dobles llaves {{ }}).
      Esto me permite traer datos que he definido en el archivo .ts y mostrarlos dinámicamente.
      He añadido iconos descriptivos para cada campo.
    -->

    <ion-item>

      <ion-icon name="person-circle-outline" slot="start" color="primary"></ion-icon>
      <ion-label>
        <h2>Nombre</h2>
```

```

        <p>{{ estudiante.nombre }}</p>
    </ion-label>
</ion-item>

<ion-item>
    <ion-icon name="school-outline" slot="start" color="secondary"></ion-icon>
    <ion-label>
        <h2>Carrera</h2>
        <p>{{ estudiante.carrera }}</p>
    </ion-label>
</ion-item>

<ion-item>
    <ion-icon name="calendar-number-outline" slot="start" color="tertiary"></ion-icon>
    <ion-label>
        <h2>Semestre</h2>
        <p>{{ estudiante.semestre }}</p>
    </ion-label>
</ion-item>
</ion-list>

<h3 class="ion-margin-top ion-padding-start">Mis Habilidades</h3>
<div class="skills-container">
    <!--
    He programado esta parte usando la directiva estructural *ngFor.
    Esta es una de las herramientas más potentes de Angular porque me permite
    repetir un elemento HTML por cada elemento que tenga en mi arreglo de habilidades.
    -->

    <ion-chip *ngFor="let hab of estudiante.habilidades" color="success" outline="true">
        <ion-icon name="checkmark-done-circle-outline"></ion-icon>
        <ion-label>{{ hab }}</ion-label>
    </ion-chip>
</div>
</ion-content>

```

BACKEND

```
import { Component } from '@angular/core';
```

```

@Component({
  selector: 'app-informacion',
  templateUrl: 'informacion.page.html',
  styleUrls: ['informacion.page.scss'],
  standalone: false,

```

```

    })
    export class InformacionPage {
        /**
         * En este punto, he definido un objeto llamado 'estudiante'.
         * He preferido agrupar todos mis datos aquí en lugar de tener variables sueltas.
         * Esto facilita la lectura del código y el mantenimiento si quiero agregar más campos.
         */
        estudiante = {
            nombre: 'Tu Nombre Aquí',
            carrera: 'Tu Carrera Universitaria',
            semestre: 'X Semestre',
            habilidades: [
                'Desarrollo con Ionic',
                'Lógica de Programación',
                'Maquetación Web (HTML/CSS)',
                'Control de Versiones (Git)'
            ]
        };

        constructor() {
            // Al igual que en la otra página, el constructor está listo por si
            // decido cargar estos datos desde una base de datos externa en el futuro.
        }
    }
}

```

Contacto

¡Hablemos!

Si tienes alguna duda sobre mi proyecto, puedes contactarme aquí.

Nombre Completo

Tu nombre

Correo Electrónico

correo@ejemplo.com

Mensaje

¿En qué puedo ayudarte?

ENVIAR FORMULARIO



Inicio



Información



Contacto

FRONTEND

```
<ion-header [translucent]="true">
```

```

<ion-toolbar color="success">
  <ion-title>
    Contacto
  </ion-title>
</ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true" class="ion-padding">
  <div class="contact-form">
    <h2 class="ion-text-center">¡Hablemos!</h2>
    <p class="ion-text-center ion-color-medium">
      Si tienes alguna duda sobre mi proyecto, puedes contactarme aquí.
    </p>

    <ion-list>
      <!--
        He utilizado el concepto de "Two-way data binding" (vinculación de datos en dos sentidos)
        usando la sintaxis [(ngModel)]. Esto significa que lo que el usuario escriba en el input
        se guardará automáticamente en mi variable del TypeScript, y viceversa.
      -->

      <ion-item>
        <ion-label position="floating">Nombre Completo</ion-label>
        <ion-input type="text" [(ngModel)]="contacto.nombre" placeholder="Tu nombre"></ion-input>
      </ion-item>

      <ion-item>
        <ion-label position="floating">Correo Electrónico</ion-label>
        <ion-input type="email" [(ngModel)]="contacto.email" placeholder="correo@ejemplo.com"></ion-input>
      </ion-item>

      <ion-item>
        <ion-label position="floating">Mensaje</ion-label>
        <ion-textarea [(ngModel)]="contacto.mensaje" placeholder="¿En qué puedo ayudarte?" rows="4"></ion-
textarea>
      </ion-item>
    </ion-list>

    <!--
      Este botón utiliza el concepto de "Event Binding" (vinculación de eventos).
      Al hacer click (click), se dispara el método enviarMensaje() que definí en mi lógica.
    -->

    <ion-button expand="block" class="ion-margin-top" color="success" (click)="enviarMensaje()">
      <ion-icon name="paper-plane-outline" slot="end"></ion-icon>

      Enviar Formulario
    </ion-button>

```

```
</div>
</ion-content>
```

BACKEND

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-contacto',
  templateUrl: 'contacto.page.html',
  styleUrls: ['contacto.page.scss'],
  standalone: false,
})
export class ContactoPage {
  /**
   * Este objeto 'contacto' es mi modelo de datos para el formulario.
   * Inicializo los campos vacíos para que el usuario pueda llenarlos.
   */
  contacto = {
    nombre: '',
    email: '',
    mensaje: ''
  };

  constructor() {}

  /**
   * He programado este método para que se ejecute cuando el usuario presiona 'Enviar'.
   * Primero imprimo los datos en la consola para verificar que se capturaron bien.
   * Luego muestro una alerta al usuario y limpio el formulario.
   */
  enviarMensaje() {
    console.log('Capturando datos del formulario:', this.contacto);

    if (this.contacto.nombre && this.contacto.email) {
      alert('¡Gracias ' + this.contacto.nombre + '! He recibido tu mensaje.');
```

```
// Limpio los campos del objeto para que el formulario quede vacío nuevamente
```

```
this.contacto = {
  nombre: '',
  email: '',
  mensaje: ''
};
```



```
    } else {  
      alert('Por favor, completa al menos tu nombre y correo.');
```

Aquí el link de mi trabajo

<https://github.com/yesguz/ionic-modular-interface>